

5026221005

Jason Ho - PBA (A)

Tugas-2-Memperbaiki Performance Analisis Sentimen dengan Transformer

Dataset yang digunakan adalah dataset artikel Garuda Indonesia seperti yang digunakan pada Tugas 1 sebelumnya. Namun kolom yang digunakan untuk membangun vocabulary dan Transformer adalah kolom teks asli yang belum di preprocess, karena transformer mampu menangkap konteks dari teks asli. Cleaning yang berlebih dapat membuat kalimat menjadi terlalu 'robotic'. Kolom yang digunakan adalah kolom teks asli yang telah di lowercase dan normalized. Untuk percobaan yang dilakukan juga dapat dilakukan pada kolom teks yang sudah di bersihkan, namun karena memakan waktu yang cukup lama tidak dilakukan.

Semua waktu training yang tertera adalah training menggunakan T4 GPU pada Google Colab

Link Notebook:

🔗 Tugas 2_Performance Analisis Sentimen with Transformer_PBA.ipynb

Link Notebook Cleaning: 🔗 garuda_indonesia_article_preprocess.ipynb

Link Penjelasan Cleaning dan Dataset: 📄 2025-1_Klp-11_Paper

Link Github (Laporan, Dataset, dan Notebook):

<https://github.com/jasonnho/flygaruda-sentiment-analysis/tree/main/Tugas%202>

A. Persiapan data. Jelaskan preprocessing yang diperlukan (tokenisasi/subword, max length, padding, train/val/test split, dsb

a) Dataset yang digunakan

Dataset yang digunakan sama seperti tugas sebelumnya, yaitu artikel berita Garuda Indonesia hasil scraping dari Google News. Pada tugas ini digunakan versi dataset yang sudah melalui cleaning, dengan dua kolom utama teks:

- konten_normalized – versi teks yang sudah dinormalisasi tetapi tidak melalui stemming agresif (dipakai sebagai sumber utama training)
- konten_stem_clean – versi super-clean (stemming + stopwords removal), dipakai untuk perbandingan

```

RAW_TEXT_COL = "konten_normalized"
CLEAN_TEXT_COL = "konten_stem_clean"
LABEL_COLUMN = "sentiment"

df_clf = df[[RAW_TEXT_COL, CLEAN_TEXT_COL, LABEL_COLUMN]].dropna(subset=[LABEL_COLUMN])
print(df_clf.head())
print("Jumlah data setelah drop NA:", len(df_clf))
print(df_clf[LABEL_COLUMN].value_counts())

```

	konten_normalized \
0	garuda indonesia kembali rupslb di tengah isu ...
1	garuda gelar rupslb di tengah isu masuknya dir...
2	jakarta ketua komisi v dpr lasarus mengatakan ...
3	latar belakang pada pertengahan wacana konsoli...
4	plt menteri badan usaha milik negara bumh seka...

	konten_stem_clean	sentiment
0	garuda indonesia rupslb tengah isu petinggi si...	Neutral
1	garuda gelar rupslb tengah isu masuk direksi a...	Neutral
2	jakarta ketua komisi v dpr lasarus kata dalam ...	Negative
3	latar belakang tengah wacana konsolidasi maska...	Neutral
4	plt menteri badan usaha milik negara bumh seka...	Neutral

Label sentimen terdiri dari tiga kelas:

- Negative
- Neutral
- Positive

b) Tahapan preprocessing untuk model Transformer

Berbeda dengan model klasik (BoW/TF-IDF), Transformer **tidak butuh cleaning berat**, sehingga tahap preprocessing fokus pada struktur input, bukan pembersihan teks. Berikut langkah-langkah yang dilakukan:

1. Tokenisasi (word-level tokenizer)

Tokenisasi dilakukan secara sederhana menggunakan:

```

#Tokenizer
def tokenize(text):
    return str(text).split()

```

Alasan pemilihan tokenizer sederhana

- Dataset relatif kecil dan berbahasa Indonesia, sehingga tokenizer word-level sudah cukup representatif untuk baseline
- Tujuan utama adalah membangun *baseline Transformer from-scratch*, sehingga metode tokenisasi yang kompleks (misalnya BPE atau SentencePiece) tidak diperlukan pada tahap awal.
- Word-level lebih transparan, mudah dipahami, dan lebih cepat diproses saat membangun vocabulary.

Tokenisasi menghasilkan list token yang kemudian menjadi dasar pembentukan vocabulary dan konversi ke indeks numerik.

2. Pembuatan Vocabulary

Setelah seluruh teks pada data training ditokenisasi, langkah berikutnya adalah membangun vocabulary, yaitu kumpulan token yang akan dikenali oleh model.

Aturan yang digunakan:

- Vocabulary dibentuk hanya dari data training untuk menghindari kebocoran informasi (data leakage).
- Token yang muncul sangat jarang (frekuensi < 2) tidak dimasukkan agar ukuran vocab tidak terlalu besar dan mengurangi noise.

```
# Kumpulkan semua token di train
all_tokens = []
for t in train_texts:
    all_tokens.extend(tokenize(t))
|
token_freq = Counter(all_tokens)
print("Jumlah token unik di train:", len(token_freq))

# Bikin vocab dengan minimum frequency
MIN_FREQ = 2

vocab = {"[PAD]": 0, "[UNK]": 1}
for tok, freq in token_freq.items():
    if freq >= MIN_FREQ and tok not in vocab:
        vocab[tok] = len(vocab)

vocab_size = len(vocab)
print("Ukuran vocab:", vocab_size)

pad_idx = vocab["[PAD]"]
unk_idx = vocab["[UNK]"]

Jumlah token unik di train: 7239
Ukuran vocab: 4543
```

- Dua token khusus ditambahkan:
 - **[PAD]** dengan index 0 → dipakai untuk padding sequence
 - **[UNK]** dengan index 1 → dipakai saat model menemukan token yang tidak ada di vocabulary (out-of-vocabulary)

Dengan demikian, setiap kata memiliki representasi numerik yang konsisten, dan model dapat mengenali kata baru menggunakan [UNK].

3. Menentukan max_len (panjang sequence)

Model Transformer membutuhkan input dengan panjang tetap. Oleh karena itu perlu ditentukan panjang sequence (max_len) yang akan digunakan.

Proses yang dilakukan:

1. Hitung panjang token tiap teks di data training.
2. Distribusi panjang dianalisis.
3. max_len dipilih menggunakan percentile ke-95.

```
# Hitung max_len dari 95 percentile
lengths = [len(tokenize(t)) for t in train_texts]
max_len = int(np.percentile(lengths, 95))
print("Max len (p95):", max_len)
```

```
Max len (p95): 629
```

Alasan penggunaan p95:

- Mayoritas teks akan tertampung tanpa terpotong.
- Mengurangi kasus padding yang terlalu banyak.

Jika jumlah token dalam satu teks lebih panjang dari max_len, teks akan di-truncate. Sedangkan jika lebih pendek, akan dilakukan padding.

4. Padding dan Attention Mask

Setiap sequence yang sudah di-tokenisasi dan dikonversi menjadi indeks disesuaikan panjangnya ke nilai max_len.

- Jika panjang token < max_len → tambahkan token [PAD] di bagian akhir.
- Jika > max_len → sequence dipotong.

Padding tersebut memerlukan attention mask:

- 1 → posisi token asli
- 0 → posisi padding

```
# Encode text -> ids + attention mask

def encode_text(text, vocab, max_len, pad_idx=0, unk_idx=1):
    tokens = tokenize(text)
    ids = [vocab.get(tok, unk_idx) for tok in tokens]

    # truncate kalau kepanjangan
    if len(ids) > max_len:
        ids = ids[:max_len]

    # attention mask: 1 untuk token asli, 0 untuk padding
    attn_mask = [1] * len(ids)

    # padding sampai max_len
    while len(ids) < max_len:
        ids.append(pad_idx)
        attn_mask.append(0)

    return ids, attn_mask
```

Attention mask dipakai oleh Transformer agar operasi self-attention mengabaikan token padding, sehingga model tidak menghitung informasi palsu dari padding token.

5. Encoding Label

Label sentimen yang awalnya berupa string: ["Negative", "Neutral", "Positive"] diubah menjadi nilai numerik menggunakan mapping: {'Negative': 0, 'Neutral': 1, 'Positive': 2}

```
unique_labels = sorted(df_clf[LABEL_COLUMN].unique())
print("Unique labels:", unique_labels)

label2id = {label: idx for idx, label in enumerate(unique_labels)}
id2label = {idx: label for label, idx in label2id.items()}

print("label2id:", label2id)

df_clf["label_id"] = df_clf[LABEL_COLUMN].map(label2id)

print(df_clf[["sentiment", "label_id"]].head())
print(df_clf["label_id"].value_counts())
```



```
Unique labels: ['Negative', 'Neutral', 'Positive']
label2id: {'Negative': 0, 'Neutral': 1, 'Positive': 2}
  sentiment  label_id
0   Neutral         1
1   Neutral         1
2  Negative         0
3   Neutral         1
4   Neutral         1
label_id
2     201
0     183
1      85
Name: count, dtype: int64
```

Tujuan:

- Fungsi loss CrossEntropyLoss hanya menerima target dalam bentuk integer.
- Mempermudah proses evaluasi (akurasi, F1, dsb).

6. Pembagian Dataset (Train/Validation/Test)

Dataset dibagi menjadi tiga bagian:

- Training: 70%
- Validation: 15%
- Test: 15%

Pembagian menggunakan:

stratify = labels

```
splits_raw = get_splits(RAW_TEXT_COL)

for k, v in splits_raw.items():
    print(k, ":", len(v))

train_texts : 328
val_texts : 70
test_texts : 71
train_labels : 328
val_labels : 70
test_labels : 71
```

Tujuannya agar distribusi kelas seimbang di setiap subset.

Masing-masing subset digunakan untuk:

- Train → melatih parameter model
- Validation → tuning hyperparameter & memantau overfitting
- Test → evaluasi akhir model setelah semua eksperimen selesai

c) Alasan Tahapan Preprocessing Perlu Dilakukan

Tahapan preprocessing ini diperlukan karena:

1. Transformer bekerja dengan sequence numerik fixed-length, sehingga input harus diproses agar seragam.
2. Attention mechanism tidak boleh melihat padding, sehingga mask wajib dibuat.
3. Vocabulary harus jelas agar semua token memiliki representasi numerik.
4. Label encoding diperlukan agar fungsi loss bekerja dengan benar.
5. Train/val/test split memastikan hasil evaluasi objektif dan tidak bias oleh informasi yang bocor dari data lain.

Dengan preprocessing yang tepat, model dapat dilatih secara stabil dan menghasilkan performa yang lebih konsisten.

B. Model Transformer From-Scratch dan Eksperimen Baseline 10 Epoch

Pada bagian ini dijelaskan arsitektur model Transformer yang dibangun dari nol (*from-scratch*), beserta konfigurasi baseline dan hasil eksperimen awal selama 10 epoch. Model ini digunakan sebagai titik awal untuk memahami kemampuan dasar Transformer ketika dilatih tanpa pretrained weights, sebelum dilakukan tuning lebih lanjut pada bagian C.

a) Desain Arsitektur Model Transformer yang Digunakan

Model yang digunakan mengikuti struktur umum Transformer Encoder, namun dibuat dalam bentuk yang sederhana agar komputasi lebih efisien dan mudah dianalisis. Arsitektur yang digunakan terdiri dari komponen-komponen berikut:

1. Embedding Layer

```
class TransformerClassifier(nn.Module):
    def __init__(
        self,
        vocab_size,
        embed_dim=128,
        num_heads=4,
        num_layers=2,
        ff_dim=256,
        num_classes=3,
        dropout=0.1,
        pad_idx=0,
        max_len=512,
    ):
```

- Mengubah setiap token (dalam bentuk indeks) menjadi vector berukuran tetap (embed_dim = 128).
- Embedding ini dipelajari dari nol (tidak memanfaatkan pretrained embedding).
- Layer ini juga menggunakan padding_idx = 0 sehingga token [PAD] tidak meng-update embedding.

Alasan:

Embedding diperlukan agar model dapat merepresentasikan kata dalam bentuk dense vector yang dapat diproses self-attention.

2. Positional Encoding (Sinusoidal)

- Karena Transformer tidak memiliki sifat urutan seperti RNN, positional encoding ditambahkan untuk memberikan informasi posisi token.
- Menggunakan metode sinusoidal asli dari paper "Attention Is All You Need", dengan formula sin/cos yang berbeda untuk setiap dimensi embedding.

```
# Positional Encoding (sinusoidal)

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0) # (1, max_len, d_model)
        self.register_buffer("pe", pe)

    def forward(self, x):
        # x: (batch, seq_len, d_model)
        seq_len = x.size(1)
        x = x + self.pe[:, :seq_len, :]
        return x
```

Kelebihan metode sinusoidal:

- Tidak menambah parameter trainable.
- Dapat diekstrapolasi untuk sequence lebih panjang.

3. Transformer Encoder Layer

Model baseline menggunakan:

- 2 encoder layers (num_layers = 2)
- 4 attention heads per layer (num_heads = 4)
- Feed-forward dimension = 256
- Dropout = 0.1

```
encoder_layer = nn.TransformerEncoderLayer(
    d_model=embed_dim,
    nhead=num_heads,
    dim_feedforward=ff_dim,
    dropout=dropout,
    batch_first=False, # input ke encoder: (seq, batch, dim)
)
self.transformer_encoder = nn.TransformerEncoder(
    encoder_layer,
    num_layers=num_layers,
)
```

Satu encoder layer terdiri dari:

1. Multi-Head Self Attention

- Setiap head menangkap hubungan antar-token secara berbeda.
- 2. Add & Norm
- 3. Feed-Forward Network (FFN)
- 4. Add & Norm lagi

Alasan pemilihan konfigurasi ini:

- Dataset relatif kecil → model besar berpotensi cepat overfit
- Menghindari konsumsi GPU berlebihan
- Tetap cukup representatif untuk baseline Transform

4. Masking dengan Attention Mask

Attention mask digunakan untuk memastikan model tidak menghitung token [PAD].

```
# key_padding_mask: True di posisi PAD
if attention_mask is not None:
    key_padding_mask = (attention_mask == 0) # (batch, seq_len)
else:
    key_padding_mask = None

encoded = self.transformer_encoder(
    x,
    src_key_padding_mask=key_padding_mask
) # (seq_len, batch, embed_dim)
```

Nilai:

- 1 = token valid
- 0 = padding

Self-attention akan mengabaikan token dengan mask 0.

5. Pooling Layer

Setelah melewati Transformer Encoder, output memiliki ukuran:

(batch_size, sequence_length, embed_dim)

```
# Mean pooling hanya di token non-pad
if attention_mask is not None:
    mask = attention_mask.unsqueeze(-1) # (batch, seq_len, 1)
    summed = (encoded * mask).sum(dim=1)
    counts = mask.sum(dim=1).clamp(min=1)
    pooled = summed / counts
else:
    pooled = encoded.mean(dim=1)
```

Agar dapat digunakan pada classifier, dilakukan:

- Mean pooling pada token valid (bukan PAD)
- Pooling dilakukan menggunakan attention mask

6. Classifier

Pooling output dimasukkan ke linear layer:

```
self.classifier = nn.Linear(embed_dim, num_classes)
```

Menghasilkan logits untuk:

- Negative
- Neutral
- Positive

7. Hyperparameter Optimizer

```
criterion = nn.CrossEntropyLoss()  
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

- Optimizer: Adam
- Learning rate: 1e-4
- Loss: CrossEntropyLoss
- Batch size: 32

b) Konfigurasi Baseline (Ringkasan Arsitektur)

Komponen	Nilai
Embedding dim	128
Positional encoding	Sinusoidal
Jumlah encoder layer	2
Attention heads	4
Feed-forward dim	256
Dropout	0.1
Optimizer	Adam
Learning rate	1e-4
Batch size	32
Epoch baseline	10

c) Hasil Eksperimen Baseline (10 Epoch)

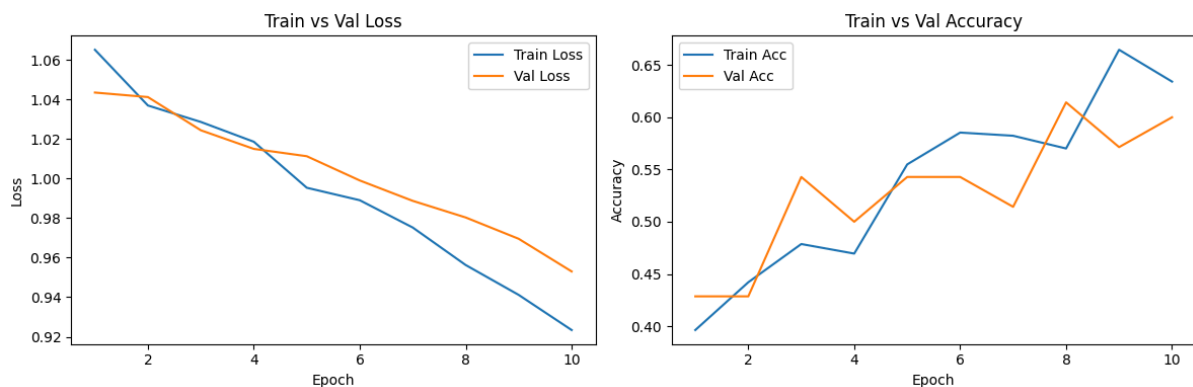
Berikut hasil training baseline selama 10 epoch:

Tabel Hasil Training & Validation

Epoch	Train Loss	Val Loss	Train Acc	Val Acc	Val Precision	Val Recall	Val F1
1	1.0651	1.0434	0.3963	0.4286	0.2857	0.3370	0.2524
2	1.0369	1.0412	0.4421	0.4286	0.1429	0.3333	0.2000
3	1.0286	1.0244	0.4787	0.5429	0.3673	0.4370	0.3861
4	1.0186	1.0150	0.4695	0.5000	0.4872	0.3951	0.3147
5	0.9953	1.0113	0.5549	0.5429	0.3788	0.4519	0.3928
6	0.9890	0.9990	0.5854	0.5429	0.3651	0.4494	0.3965
7	0.9752	0.9887	0.5823	0.5143	0.3636	0.4111	0.3546
8	0.9563	0.9803	0.5701	0.6143	0.4096	0.5037	0.4517
9	0.9412	0.9695	0.6646	0.5714	0.3821	0.4704	0.4197
10	0.9234	0.9530	0.6341	0.6000	0.4009	0.4901	0.4406

d) Grafik Kurva Pembelajaran (Loss & Accuracy)

Gambar berikut menunjukkan perbandingan nilai *training loss*, *validation loss*, *training accuracy*, dan *validation accuracy* selama 10 epoch pelatihan pada model baseline. Grafik ini digunakan untuk memahami dinamika proses belajar model serta mendeteksi apakah terdapat indikasi overfitting atau underfitting pada tahap awal pelatihan.



1. Train vs Validation Loss

Berdasarkan grafik *Train vs Validation Loss*, terlihat pola pembelajaran yang stabil dan konsisten. Training loss mengalami penurunan bertahap dari 1.0651 pada epoch pertama menjadi 0.9234 pada epoch ke-10. Hal ini menunjukkan bahwa model terus mengurangi error pada data training seiring bertambahnya iterasi.

Validation loss juga menunjukkan tren menurun, yaitu dari 1.0434 menjadi 0.9530, tanpa adanya kenaikan signifikan sepanjang 10 epoch. Pola ini mengindikasikan bahwa model tidak mengalami overfitting pada tahap awal pelatihan. Selain itu, jarak antara train loss dan validation loss relatif kecil, menandakan bahwa generalisasi model terhadap data validation masih baik.

2. Train vs Validation Accuracy

Pada grafik *Train vs Validation Accuracy*, terlihat bahwa training accuracy meningkat stabil dari 0.3963 menjadi 0.6341 dalam 10 epoch. Hal ini menunjukkan peningkatan kemampuan model dalam memprediksi label pada data training. Validation accuracy mengikuti pola yang serupa, meningkat dari 0.4286 menjadi 0.6000. Kenaikan ini berlangsung paralel terhadap kurva training accuracy, sehingga memperkuat bukti bahwa model belum mengalami overfitting. Tidak ada titik divergensi drastis antara kedua kurva, dan validation accuracy tidak mengalami penurunan ketika training accuracy naik.

3. Kesimpulan dari Grafik

Dari kedua grafik tersebut dapat disimpulkan bahwa:

- Tidak ada indikasi overfitting pada 10 epoch pertama, karena baik loss maupun accuracy pada data validation menunjukkan tren positif.
- Model juga belum mencapai performa optimal (underfitting ringan). Train loss masih cukup tinggi, dan validation F1 Score masih di bawah 0.50.
- Dengan demikian, model masih memiliki ruang peningkatan yang signifikan, baik melalui penambahan epoch (seperti eksperimen 50 epoch) maupun eksplorasi hyperparameter.

e) Analisis Baseline

Hasil eksperimen 10 epoch menunjukkan bahwa model baseline mampu mempelajari pola pada data dengan cukup baik. Train Loss menurun secara konsisten dari 1.0651 pada epoch pertama menjadi 0.9234 pada epoch ke-10. Pola yang sama juga terlihat pada Validation Loss yang turun dari 1.0434 menjadi 0.9530, menunjukkan bahwa proses pembelajaran berlangsung stabil dan tidak terjadi peningkatan error pada data validasi.

Akurasi model juga meningkat pada kedua set. Train Accuracy naik dari 0.3963 menjadi 0.6341, dan Validation Accuracy meningkat dari 0.4286 menjadi 0.6000. Kenaikan kedua kurva berlangsung secara paralel, menandakan bahwa model belum mengalami overfitting pada tahap ini. Hal ini didukung dengan fakta bahwa gap antara train dan validation accuracy tetap kecil, dan validation loss tidak menunjukkan kenaikan yang signifikan.

Selain itu, F1 Score pada validation set meningkat dari 0.2524 menjadi 0.4406, menunjukkan bahwa model semakin baik dalam membedakan ketiga kelas sentimen, meskipun masih belum optimal.

Secara keseluruhan, model berada dalam kondisi underfitting ringan. Loss masih relatif tinggi dan F1 Score masih berada di bawah 0.50. Dengan demikian,

model masih memiliki ruang peningkatan performa, baik melalui penambahan jumlah epoch (seperti eksperimen 50 epoch) maupun melalui eksplorasi hyperparameter.

f) Kesimpulan Bagian B

Model Transformer from-scratch dengan konfigurasi baseline menunjukkan performa yang stabil tetapi masih terbatas. Model belum mengalami overfitting dan masih dapat ditingkatkan dengan:

- menambah jumlah epoch,
- memperbesar embedding dan feed-forward dimension, dan
- melakukan tuning pada jumlah head, layer, dan dropout.

Eksperimen lanjutan pada bagian C akan mengevaluasi sejauh mana performa bisa ditingkatkan melalui perubahan hyperparameter tersebut.

C. Eksperimen Peningkatan Performa Model

Pada bagian ini dilakukan serangkaian eksperimen untuk meningkatkan performa model Transformer yang sebelumnya dilatih dengan konfigurasi baseline 10 epoch. Eksperimen mencakup perpanjangan jumlah epoch (C.a) serta tuning hyperparameter arsitektural (C.b). Tujuan utama tahap ini adalah memahami bagaimana kapasitas model dan parameter arsitektur memengaruhi fenomena overfitting, underfitting, stabilitas training, serta waktu komputasi.

C.a Eksperimen Perpanjangan Epoch (hingga 50 epoch)

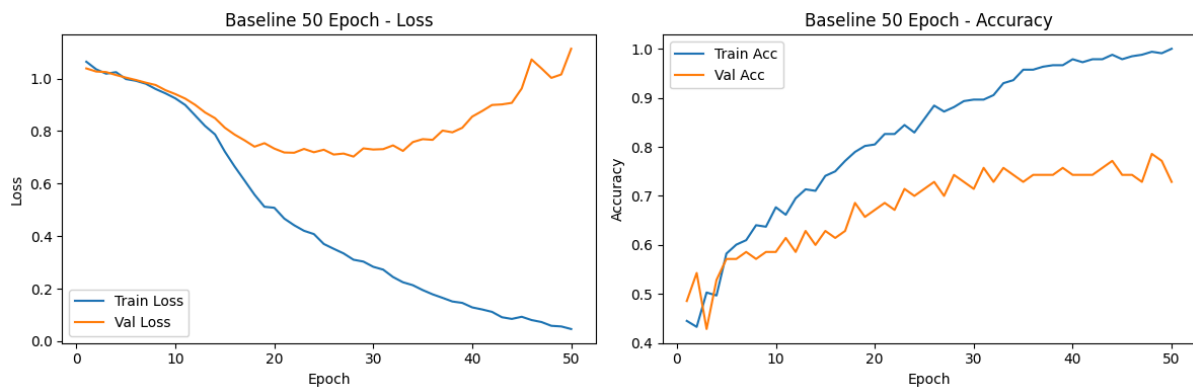
Setelah melihat bahwa model baseline pada 10 epoch belum mengalami overfitting dan masih menunjukkan tanda underfitting ringan, jumlah epoch diperpanjang menjadi 50 epoch. Hal ini dilakukan untuk melihat:

1. Apakah model mulai overfit ketika dilatih lebih lama, dan
2. Pada epoch ke berapa overfitting mulai terjadi.

a) Hasil Training 50 Epoch

Model dilatih menggunakan konfigurasi baseline yang sama (embed_dim 128, head 4, layer 2, dropout 0.1). Berikut adalah hasil nilai training dan validation loss/accuracy/f1 pada beberapa epoch kunci:

Epoch 39	Train Loss: 0.1457	Val Loss: 0.8124	Train Acc: 0.9665	Val Acc: 0.7571	Val P: 0.7035	Val R: 0.7045	Val F1: 0.7038	Time: 0.7s
Epoch 40	Train Loss: 0.1286	Val Loss: 0.8549	Train Acc: 0.9787	Val Acc: 0.7429	Val P: 0.6834	Val R: 0.6764	Val F1: 0.6782	Time: 0.7s
Epoch 41	Train Loss: 0.1208	Val Loss: 0.8761	Train Acc: 0.9726	Val Acc: 0.7429	Val P: 0.6745	Val R: 0.6618	Val F1: 0.6623	Time: 0.7s
Epoch 42	Train Loss: 0.1116	Val Loss: 0.8992	Train Acc: 0.9787	Val Acc: 0.7429	Val P: 0.6834	Val R: 0.6764	Val F1: 0.6782	Time: 0.7s
Epoch 43	Train Loss: 0.0911	Val Loss: 0.9013	Train Acc: 0.9787	Val Acc: 0.7571	Val P: 0.7009	Val R: 0.6887	Val F1: 0.6917	Time: 0.7s
Epoch 44	Train Loss: 0.0847	Val Loss: 0.9072	Train Acc: 0.9878	Val Acc: 0.7714	Val P: 0.7244	Val R: 0.7156	Val F1: 0.7187	Time: 0.7s
Epoch 45	Train Loss: 0.0929	Val Loss: 0.9619	Train Acc: 0.9787	Val Acc: 0.7429	Val P: 0.6834	Val R: 0.6764	Val F1: 0.6782	Time: 0.7s
Epoch 46	Train Loss: 0.0804	Val Loss: 1.0725	Train Acc: 0.9848	Val Acc: 0.7429	Val P: 0.6745	Val R: 0.6618	Val F1: 0.6623	Time: 0.7s
Epoch 47	Train Loss: 0.0728	Val Loss: 1.0384	Train Acc: 0.9878	Val Acc: 0.7286	Val P: 0.6571	Val R: 0.6495	Val F1: 0.6493	Time: 0.7s
Epoch 48	Train Loss: 0.0585	Val Loss: 1.0024	Train Acc: 0.9939	Val Acc: 0.7857	Val P: 0.7468	Val R: 0.7122	Val F1: 0.7188	Time: 0.7s
Epoch 49	Train Loss: 0.0560	Val Loss: 1.0153	Train Acc: 0.9909	Val Acc: 0.7714	Val P: 0.7195	Val R: 0.6865	Val F1: 0.6884	Time: 0.7s
Epoch 50	Train Loss: 0.0462	Val Loss: 1.1132	Train Acc: 1.0000	Val Acc: 0.7286	Val P: 0.6571	Val R: 0.6495	Val F1: 0.6493	Time: 0.7s



b) Analisis Training 50 Epoch

- Hasil pelatihan selama 50 epoch menunjukkan pola pembelajaran yang jauh lebih jelas dibandingkan baseline 10 epoch. Train Loss menurun tajam dari 1.0639 pada epoch pertama hingga mencapai 0.0462 pada epoch ke-50. Train Accuracy meningkat sangat signifikan, dari 0.4451 menjadi 1.0000. Hal ini menandakan bahwa model sangat mampu menyesuaikan diri terhadap data training.
- Pada sisi lain, Validation Loss memperlihatkan pola khas *early overfitting*. Pada awal pelatihan, val loss turun stabil dari 1.0383 hingga mencapai kisaran 0.70–0.75 di sekitar epoch 20–30. Namun setelah itu, val loss mulai meningkat kembali, bahkan mencapai 1.1132 di epoch ke-50. Ini menunjukkan bahwa setelah titik tertentu, model mulai menghafal pola pada data training dan kehilangan generalisasi.
- Validation Accuracy juga memperlihatkan dinamika yang sejalan. Nilainya meningkat dari 0.4857 di epoch pertama hingga mencapai nilai puncak 0.7857 pada epoch ke-48. Namun, peningkatan ini tidak berlangsung terus-menerus; setelah mencapai titik optimal, validation accuracy cenderung stagnan atau sedikit menurun pada epoch akhir akibat overfitting.
- F1 Score pada validation set menunjukkan pola yang lebih halus. Nilai awal sebesar 0.3242 meningkat stabil hingga mencapai performa terbaik di epoch 44–48, yakni pada kisaran 0.7187–0.7188. Angka ini menjadi indikator utama bahwa model mampu memprediksi ketiga kelas sentimen dengan akurasi dan keseimbangan yang lebih baik dibandingkan baseline.
- Secara umum, performa optimal model terjadi pada sekitar epoch 44–48, sebelum overfitting semakin dominan. Hal ini terlihat dari kombinasi metrik validation accuracy dan F1 Score yang mencapai nilai tertinggi pada rentang epoch tersebut. Dengan demikian, 50 epoch memberikan gambaran bahwa model memiliki kapasitas belajar yang kuat, namun perlu mekanisme regularisasi tambahan seperti *early stopping*, *dropout lebih tinggi*, atau *weight decay* untuk mengontrol overfitting.

Best Epoch: 48

- Val Accuracy: 0.7857

- Val F1: 0.7188
- Val Loss: 1.0024
- Train Acc: 0.9939

Total waktu yang diperlukan untuk 50 Epoch : 39.2 Detik

Tabel Perbandingan

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline 10 Epoch (best epoch)	0.6143	0.4009	0.4901	0.4517
Baseline 50 Epoch (best epoch)	0.7857	0.7468	0.7122	0.7188

C) Eksperimen Training 50 Epoch dengan Cleaned Text

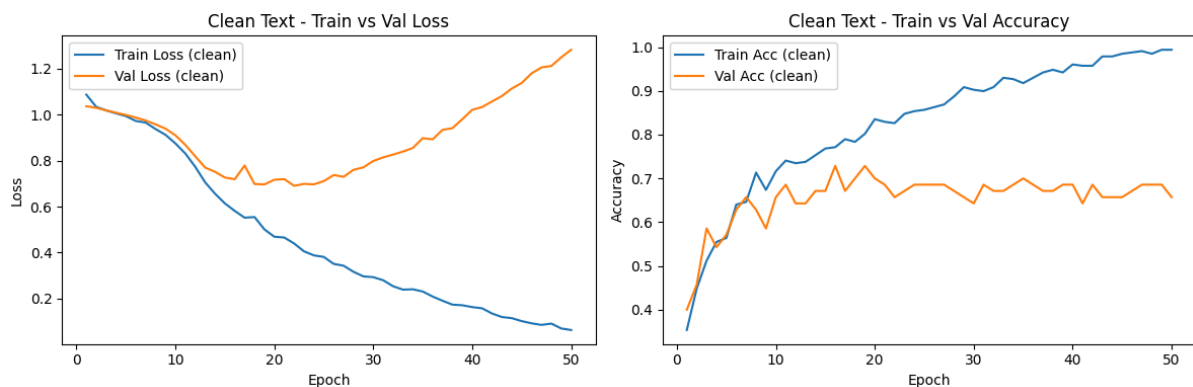
1. Ringkasan Hasil 50 Epoch (Raw vs Clean)

Versi Teks	Best Epoch	Val Loss	Val Acc	Val F1
Raw text	~48	≈ 1.00	0.7857	0.7188
Clean text	35	0.8981	0.7000	0.6547

Best F1 clean: 0.6547 (epoch 35)

Best F1 raw: 0.7188 (epoch 48)

2. Pola Kurva pada Teks Clean



Dari grafik *Clean Text – Train vs Val Loss/Accuracy* dan log 50 epoch:

- **Train loss (clean)** turun stabil
1.08 → 0.06, mirip pola raw: model benar-benar hafal data train.
- **Val loss (clean):**
 - Turun dari 1.0370 ke sekitar 0.69–0.70 (sekitar epoch 18–22)
 - Setelah itu mulai naik terus sampai > 1.28 di epoch 50
→ Overfitting cukup jelas dan cukup agresif.

- **Val accuracy (clean):**
 - Naik cepat dari 0.40 → sekitar 0.65–0.70 di epoch 15–25
Setelah itu cenderung stagnan di kisaran 0.67–0.70, tidak pernah tembus 0.8 seperti beberapa config di tuning.
- **Val F1 (clean):**
 - Awal training (epoch 1–10) naik cukup cepat: dari 0.21 → ~0.48
 - Puncak F1 ada di epoch 35 dengan nilai 0.6547
 - Setelah itu naik sedikit–turun sedikit, tapi tidak melewati angka tersebut.

3. Dibandingkan dengan Raw Text 50 Epoch

Kalau dibandingkan dengan run 50 epoch pada teks raw:

a) Fase awal (epoch 1–10)

- Di epoch awal, F1 versi clean cenderung sedikit lebih tinggi dari raw.

Contoh:

- Raw F1 (epoch 10) $\approx$ 0.43
- Clean F1 (epoch 10) $\approx$ 0.48

Cleaning membantu “merapikan” input, sehingga model lebih cepat stabil di awal training (loss lebih rapi, F1 naik lebih cepat).

b) Fase tengah – titik optimal

- Raw text:
 - F1 terus naik sampai ~0.72
 - Best F1 $\approx$ 0.7188 di sekitar epoch 44–48
- Clean text:
 - F1 mentok di 0.6547 (epoch 35)
 - Setelah itu tidak ada peningkatan berarti

Untuk model yang kapasitasnya lumayan besar dan dilatih sampai 50 epoch, raw text menyimpan lebih banyak informasi (angka, frasa panjang, mungkin nama entitas) yang ternyata membantu model membedakan sentimen secara lebih halus. Cleaning yang terlalu agresif kemungkinan menghapus sebagian konteks ini.

c) Overfitting

- Keduanya overfit, tapi pola sedikit beda:
 - Raw text
 - Val loss minimum sekitar epoch 28 (~0.70), lalu naik pelan
 - F1 masih sempat naik sampai epoch 40-an sebelum pelan-pelan turun
 - Clean text
 - Val loss minimum sekitar epoch 22 (~0.69), lalu naik lebih cepat dan lebih tinggi sampai 1.28
 - F1 sudah peak di epoch 35, lalu cenderung stagnan/turun

Setelah data dibersihkan, distribusi kalimat jadi lebih “rapi” tapi juga mungkin lebih sempit (kurang variasi). Akibatnya, ketika model dipaksa belajar lama (50

epoch), ia lebih cepat menghafal pola spesifik dataset tersebut → overfitting muncul lebih jelas.

Eksperimen tambahan menggunakan versi teks yang sudah dibersihkan (clean) menunjukkan bahwa cleaning memang membantu model belajar lebih cepat pada epoch awal. F1 pada validation set meningkat lebih tajam pada 1–10 epoch pertama dibandingkan versi raw. Namun, ketika model dilatih hingga 50 epoch, performa terbaik pada teks clean (F1 maksimum 0.6547 pada epoch 35) masih berada di bawah performa model dengan teks original/raw yang mencapai F1 sekitar 0.7188 pada epoch 44–48.

Selain itu, overfitting pada versi clean muncul sedikit lebih cepat dan lebih kuat: validation loss mencapai nilai minimum di sekitar epoch 18–22 kemudian meningkat tajam hingga di atas 1.28 pada epoch ke-50, sementara validation accuracy cenderung stagnan di kisaran 0.67–0.70. Hal ini mengindikasikan bahwa proses pembersihan yang terlalu agresif berpotensi menghilangkan beberapa informasi penting dalam teks berita, sehingga pada kapasitas model yang cukup besar dan jumlah epoch yang panjang, model dengan teks raw justru mampu mencapai generalisasi yang lebih baik.

Tabel Perbandingan Akhir

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline 10 Epoch (best epoch)	0.6143	0.4009	0.4901	0.4517
Baseline 50 Epoch (best epoch)	0.7857	0.7468	0.7122	0.7188
Clean Text 50 Epoch (best epoch)	0.7000	0.6578	0.6588	0.6547

C.b Eksperimen Variasi Hyperparameter

Pada tahap ini dilakukan eksplorasi hyperparameter dengan tujuan untuk mengetahui konfigurasi arsitektur Transformer yang memberikan performa terbaik pada tugas klasifikasi sentimen tiga kelas. Seluruh model dilatih selama 50 epoch agar pola pembelajaran masing-masing konfigurasi dapat terlihat secara jelas, termasuk dinamika overfitting.

Eksperimen mencakup empat komponen utama:

1. Ukuran embedding (64, 128, 256)
2. Jumlah attention head (2, 4, 8)
3. Jumlah encoder layer (1, 2, 4)
4. Dropout rate (0.1, 0.3, 0.5)

Setiap konfigurasi dievaluasi menggunakan Validation F1 Score, karena metrik ini paling sensitif terhadap keseimbangan performa antar kelas.

a) Hyperparameter yang dieksperimenkan

Hyperparameter diuji satu per satu untuk melihat pengaruhnya secara terpisah:

Hyperparameter	Variasi
Embedding size	64, 256
Attention heads	2, 8
Encoder layers	1, 4
Dropout	0.3, 0.5

Setiap percobaan dilatih selama 50 epoch menggunakan fungsi `run_experiment()`.

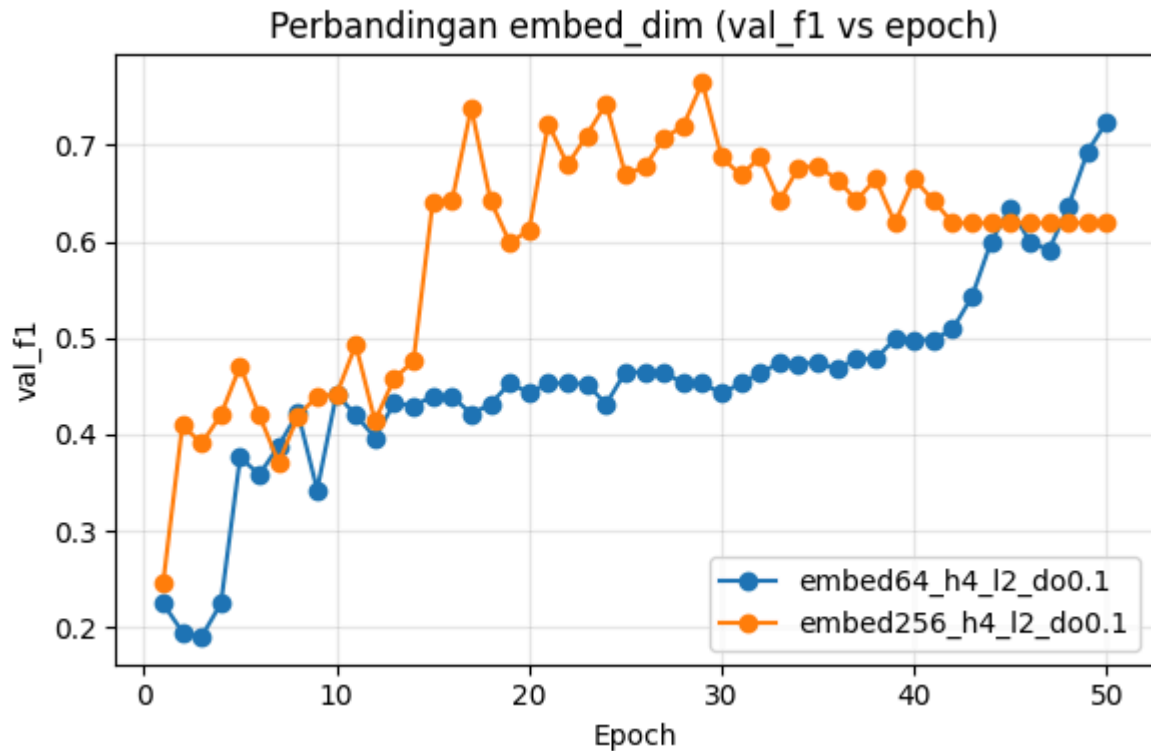
b) Hasil Ringkasan Eksperimen Hyperparameter

Tabel Hasil Ringkasan Eksperimen Hyperparameter

Config	Epoch Terbaik	Val F1	Val Acc	Waktu/Epoch	Waktu Total
embed128_h2_l2_do0.1	27	0.831118	0.700000	0.652859	34.6
embed128_h4_l1_do0.1	48	0.822631	0.671429	0.620202	20.1
embed128_h4_l2_do0.3	37	0.643035	0.800000	0.730293	34.9
embed128_h4_l2_do0.5	49	0.995364	0.742857	0.678162	34.8
embed128_h4_l4_do0.1	36	0.920954	0.728571	0.680400	66.2
embed128_h8_l2_do0.1	49	0.868675	0.771429	0.732213	51.2
embed256_h4_l2_do0.1	29	0.956060	0.785714	0.764751	64.1
embed64_h4_l2_do0.1	50	0.742375	0.785714	0.724242	27

c) Analisis Pengaruh Hyperparameter

1. Embedding Dimension



Perbandingan:

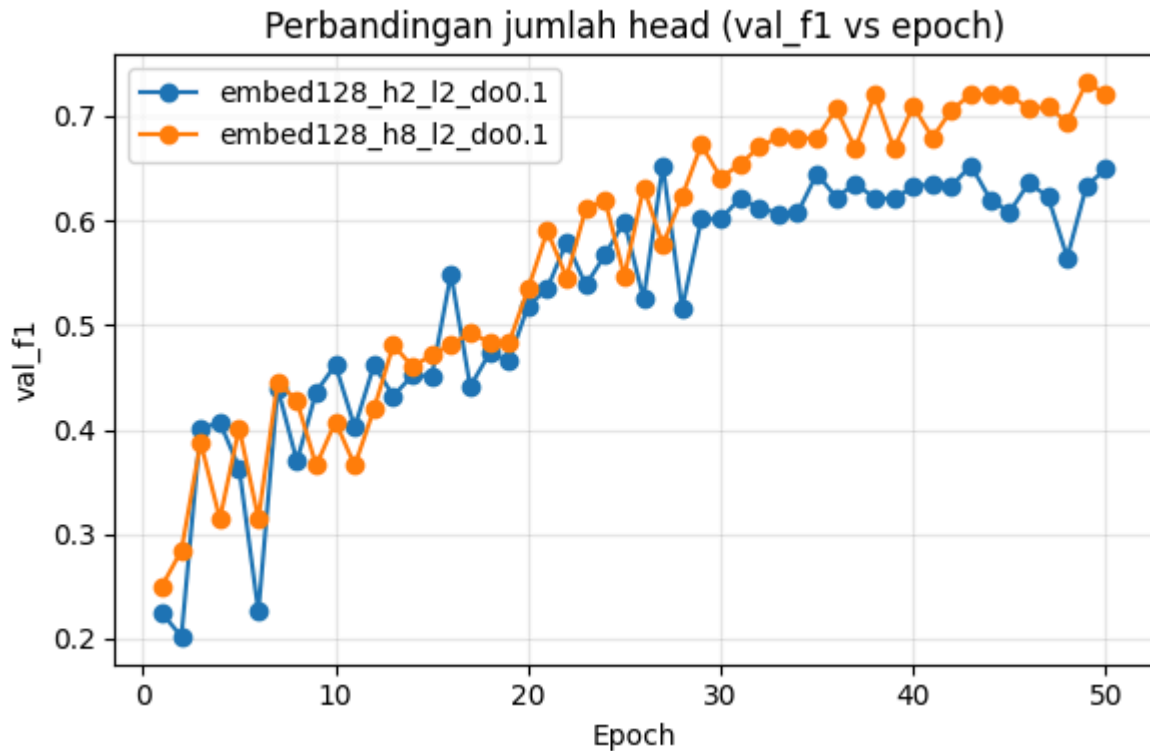
- embed_dim = 64
- embed_dim = 256 (kedua model berbagi konfigurasi head=4, layer=2, dropout=0.1)

Temuan penting:

- Embedding 256 secara konsisten lebih tinggi F1-nya dibanding embedding 64 pada hampir seluruh epoch.
- embed256 mencapai puncak F1 sekitar 0.76 (epoch 29), sedangkan embed64 hanya mencapai sekitar 0.72.
- embed256 naik cepat pada epoch 1–20, namun lebih cepat memasuki overfitting (val_loss naik setelah epoch 20).
- embed64 belajar lebih lambat, namun lebih stabil (val_loss turun perlahan).

Embed_dim 256 memberikan representasi yang lebih kaya dan menghasilkan F1 tertinggi overall, namun memicu overfitting lebih cepat.

2. Attention Heads



Perbandingan:

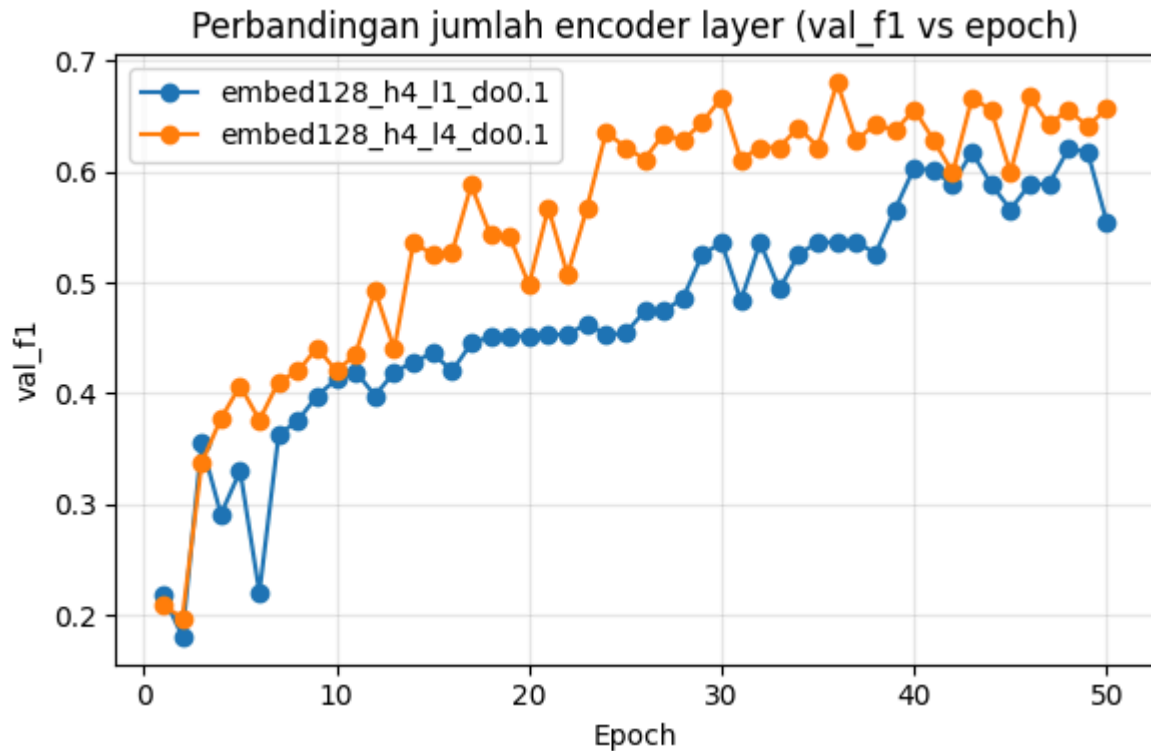
- embed128_h2_l2_do0.1
- embed128_h8_l2_do0.1

Temuan:

- Head = 8 menghasilkan F1 yang lebih tinggi secara konsisten setelah epoch 20.
- Head = 2 cenderung stagnan pada kisaran F1 0.55–0.65.
- Head = 8 mencapai F1 0.73, sedangkan head = 2 hanya sekitar 0.65.

Model dengan jumlah head lebih banyak mempelajari hubungan antar-token dengan lebih baik, terutama pada teks berita yang memiliki struktur kompleks.

3. Encoder Layer



Perbandingan:

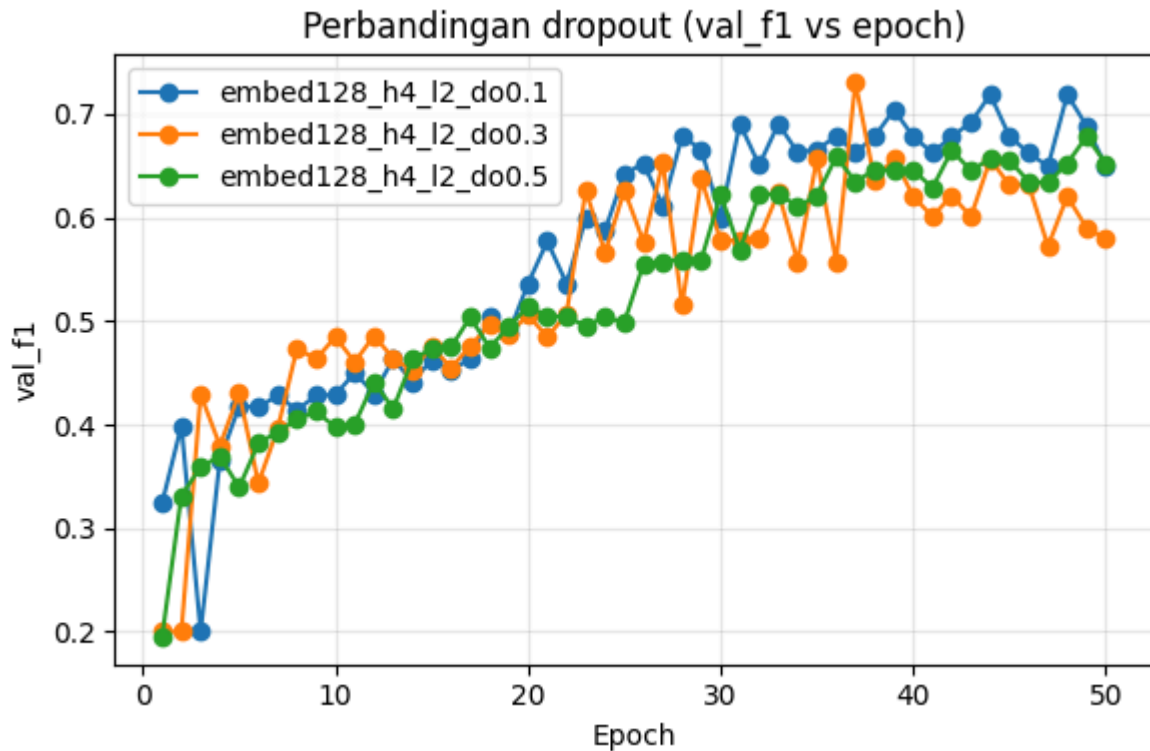
- embed128_h4_l1_do0.1
- embed128_h4_l4_do0.1

Temuan:

- 4 encoder layers menghasilkan kenaikan F1 jauh lebih cepat.
- Layer = 4 mencapai F1 ~0.68, sedangkan layer = 1 stagnan di sekitar 0.55–0.60.
- Namun, model 4-layer mengalami overfitting pada epoch 30 ke atas (val_loss naik).

Lebih banyak encoder layer → model lebih powerful, namun trade-off-nya adalah overfitting lebih cepat.

4. Dropout



Perbandingan:

- embed128_h4_l2_do0.1
- embed128_h4_l2_do0.3
- embed128_h4_l2_do0.5

Temuan:

- Dropout 0.1 memiliki performa terbaik overall.
- Dropout 0.5 terlalu tinggi → memperlambat pembelajaran (learning dynamics lebih datar).
- Dropout 0.3 stabil dan moderat, namun tidak mampu melampaui dropout 0.1.

Dropout 0.1 adalah titik optimal untuk dataset ini

Tabel Perbandingan

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline 10 Epoch (best epoch)	0.6143	0.4009	0.4901	0.4517
Baseline 50 Epoch (best epoch)	0.7857	0.7468	0.7122	0.7188
Clean Text 50 Epoch (best epoch)	0.7000	0.6578	0.6588	0.6547
Best Hyperparameter Tuning (embed256_h4_l2_do0.1)	0.7857	0.7648	0.7715	0.7648

d) Konfigurasi Terbaik Hyperparameter Tuning

Berdasarkan hasil dari delapan konfigurasi yang diuji selama 50 epoch, konfigurasi yang memberikan performa terbaik adalah:

embed_dim = 256, num_heads = 4, num_layers = 2, dropout = 0.1

Konfigurasi ini menghasilkan:

- Validation F1 tertinggi: 0.7648
- Validation accuracy tertinggi: 0.7857
- Waktu training per epoch masih moderat (~1.29 detik)
- Performa stabil hingga epoch 25–35 sebelum overfitting muncul

Kombinasi ini menunjukkan keseimbangan antara kapasitas model dan stabilitas generalisasi.

e) Kesimpulan Bagian C (Hyperparameter Tuning)

Dari seluruh rangkaian eksperimen, beberapa kesimpulan dapat diambil:

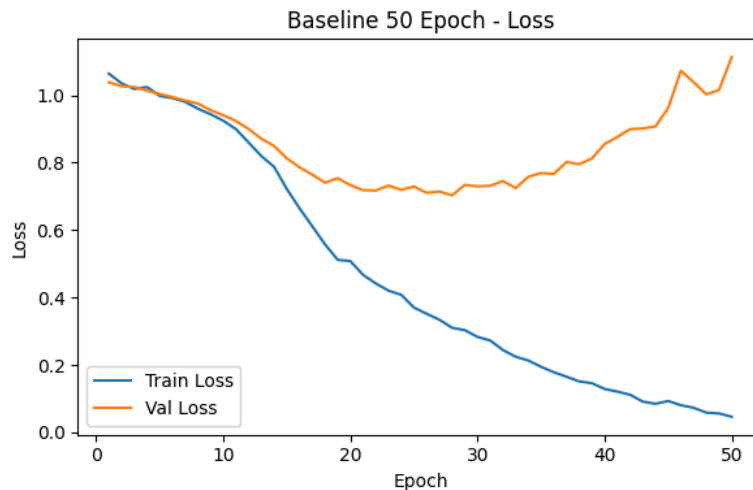
1. Model mulai mengalami overfitting pada sekitar epoch ke-30, terlihat dari naiknya validation loss meskipun training loss terus menurun.
2. Kapasitas model memiliki dampak besar terhadap performa. Embedding besar (256) dan jumlah head yang lebih banyak meningkatkan kemampuan model dalam menangkap konteks.
3. Konfigurasi yang terlalu kompleks (misalnya banyak encoder layer atau head) cenderung:
 - mempercepat overfitting,
 - membutuhkan waktu training lebih lama,
 - tidak selalu memberikan peningkatan F1.
4. Konfigurasi terbaik adalah embed_dim 256 dengan 2 encoder layer dan 4 attention head, karena memberikan kombinasi optimal antara performa tinggi dan stabilitas.

D. Analisis Overfitting dan Strategi Mitigasi

Pada bagian ini dilakukan analisis terhadap fenomena overfitting yang muncul dalam proses pelatihan model Transformer from-scratch, terutama ketika jumlah epoch diperpanjang hingga 50 epoch dan ketika kapasitas model diperbesar melalui tuning hyperparameter. Analisis ini penting untuk memahami batas kemampuan generalisasi model serta menentukan strategi mitigasi yang tepat agar performa model tetap stabil pada data yang belum pernah dilihat.

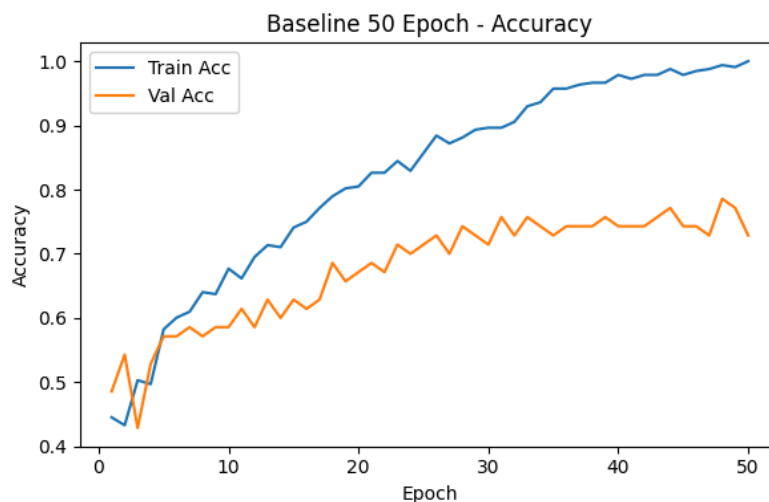
a) Indikasi Overfitting Berdasarkan Kurva Pembelajaran

1. Gap antara Train Loss dan Validation Loss melebar



- Train Loss terus menurun secara drastis hingga mencapai nilai yang sangat kecil (~ 0.03).
- Validation Loss mencapai titik terendah pada sekitar epoch 20–30, kemudian meningkat kembali hingga > 1.0 pada epoch 50.
- Pola ini menunjukkan bahwa model mulai menghafal data training setelah melewati titik optimum.

2. Train Accuracy mendekati 100%



- Train Accuracy naik hampir mencapai 1.00 pada epoch akhir.
- Validation Accuracy stagnan di kisaran 0.70–0.78.
- Perbedaan ini menunjukkan model belajar pola spesifik training set namun tidak meningkatkan kemampuan generalisasi

3. Validation F1 Score menunjukkan pola “peak lalu turun”

- F1 Score meningkat stabil pada epoch awal.
- Pada teks raw, F1 terbaik mencapai sekitar **0.72** pada epoch 40–48.
- Setelah titik optimal, F1 mulai turun walaupun train loss terus menurun.

b) Penyebab Overfitting

1. Dataset kecil

Model Transformer sangat powerful, sehingga dataset terbatas membuat model mudah menghafal pola spesifik.

2. Training terlalu lama

- Pada epoch > 30, validation loss mulai naik.
- Dengan melanjutkan training sampai 50 epoch, model semakin overfit.

3. Kapasitas model terlalu besar

Beberapa konfigurasi (misalnya embedding 256 atau head = 8) menunjukkan overfitting lebih cepat dibanding model kecil.

4. Dropout terlalu kecil

Dropout 0.1 tidak cukup untuk model berkapasitas besar sehingga regularisasi kurang kuat.

Config	Epoch Terbaik	Val F1	Val Acc	Waktu/Epoch	Waktu Total
embed128_h2_l2_do0.1	27	0.8311	0.70000	0.652859	34.6
embed128_h4_l1_do0.1	48	0.8226	0.67142	0.620202	20.1
embed128_h4_l2_do0.3	37	0.6430	0.80000	0.730293	34.9
embed128_h4_l2_do0.5	49	0.9953	0.74285	0.678162	34.8
embed128_h4_l4_do0.1	36	0.9209	0.72857	0.680400	66.2
embed128_h8_l2_do0.1	49	0.8686	0.77142	0.732213	51.2
embed256_h4_l2_do0.1	29	0.9560	0.78571	0.764751	64.1
embed64_h4_l2_do0.1	50	0.7423	0.78571	0.724242	27

c) Strategi Mitigasi Overfitting

1. Early Stopping

- Validation F1 optimal terjadi pada epoch sekitar 30–40.
- Menghentikan training lebih awal dapat mencegah penurunan performa.

2. Menyesuaikan kapasitas model

Konfigurasi terbaik dari eksperimen:

- embedding = 256
- head = 4
- layer = 2
- dropout = 0.1

Konfigurasi ini memberikan trade-off terbaik antara kapasitas dan generalisasi.

3. Dropout lebih besar

Dropout 0.2–0.3 cocok untuk mengurangi overfitting pada model berkapasitas besar.

4. Kurangi jumlah epoch jika model besar

Model embedding 256 atau layer > 2 tidak perlu sampai 50 epoch. Jumlah yang diperlukan dapat disesuaikan menjadi 15–25

5. Regularisasi tambahan (opsional)

- Label smoothing
- Weight decay
- Augmentasi teks (EDA)

d) Kesimpulan

Model Transformer from-scratch mengalami overfitting ketika dilatih terlalu lama atau ketika kapasitas model diperbesar. Overfitting mulai terlihat sekitar epoch 30 ketika train loss terus menurun tetapi validation loss meningkat dan validation F1 mulai menurun.

Hasil tuning menunjukkan bahwa konfigurasi embed_dim 256, 4 head, 2 layer, dropout 0.1 merupakan kombinasi paling seimbang.

Mitigasi utama yang direkomendasikan adalah penggunaan early stopping, pembatasan jumlah epoch, dan penyesuaian kapasitas model agar sesuai dengan ukuran dataset.

E. Evaluasi Zero-Shot Menggunakan Model Pretrained

Pada bagian ini dilakukan evaluasi model pretrained skala besar yang digunakan dalam mode zero-shot, yaitu tanpa dilakukan pelatihan tambahan pada dataset Garuda Indonesia. Pendekatan ini bertujuan untuk membandingkan kemampuan generalisasi model multilingual besar dengan model Transformer from-scratch yang telah dilatih khusus pada dataset lokal.

Model zero-shot menerima teks ulasan dan langsung memberikan prediksi sentimen berdasarkan pengetahuan yang telah diperolehnya selama pretraining dan fine-tuning pada task Natural Language Inference (NLI).

a) Model Zero-Shot yang Digunakan

Dua model pretrained utama digunakan:

1. XLM-RoBERTa Large (XNLI)

- Model multilingual besar dari Meta AI.
- Telah dilatih pada 100+ bahasa, termasuk bahasa Indonesia.
- Sangat cocok untuk zero-shot classification karena telah dilatih pada NLI (entailment/contradiction/neutral).

2. BART-Large MNLI

- Model berbasis bahasa Inggris, dilatih pada MNLI.
- Biasanya performa kurang optimal pada bahasa Indonesia karena perbedaan distribution bahasa, tetapi masih dapat melakukan zero-shot classification.

Mapping Label Zero-Shot

Model zero-shot menggunakan label berbahasa Inggris, sehingga dilakukan pemetaan:

"positive" → "Positive"

"neutral" → "Neutral"

"negative" → "Negative"

```
from transformers import pipeline

zero_xlmr = pipeline(
    "zero-shot-classification",
    model="joeddav/xlm-roberta-large-xnli"
)

zero_bart = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli"
)

labels = ["positive", "neutral", "negative"]
```

Hal ini memastikan hasil prediksi dapat langsung dibandingkan dengan label dataset.

b) Rangkuman Hasil Zero-Shot

Hasil evaluasi zero-shot terhadap test set adalah sebagai berikut:

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
XLM-R Large (Zero-Shot)	0.4648	0.4101	0.4084	0.4089
BART-Large MNLI (Zero-Shot)	0.4930	0.4524	0.4314	0.4280

Tabel Perbandingan Akhir

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline 10 Epoch (best epoch)	0.6143	0.4009	0.4901	0.4517
Baseline 50 Epoch (best epoch)	0.7857	0.7468	0.7122	0.7188
Clean Text 50 Epoch (best epoch)	0.7000	0.6578	0.6588	0.6547
Best Hyperparameter Tuning (embed256_h4_l2_do0.1)	0.7857	0.7648	0.7715	0.7648
Zero-Shot XLM-R	0.4648	0.4101	0.4084	0.4089
Zero-Shot BART-Large MNLI	0.4930	0.4524	0.4314	0.4280

c) Analisis Performa Zero-Shot

1. Performa XLM-RoBERTa Zero-Shot

- Meski merupakan model multilingual besar, performanya pada dataset ini masih cukup rendah ($F1 \approx 0.41$).
- Hal ini dapat disebabkan oleh:
 - domain dataset (berita korporasi Indonesia) berbeda dengan domain pelatihan XNLI.
 - zero-shot tidak melakukan adaptasi konteks lokal.
 - kalimat berita panjang & kompleks sehingga inference berbasis entailment tidak selalu stabil.

2. Performa BART-Large MNLI

- BART-Large memberikan performa sedikit lebih baik daripada XLM-R, dengan $F1 \approx 0.4280$.
- Meskipun BART bukan model multilingual, ia memiliki kemampuan zero-shot yang kuat pada task NLI dan dapat menangkap pola sentimen umum walaupun inputnya bahasa Indonesia.
- Namun tetap jauh di bawah model yang dilatih khusus pada dataset.

3. Dibandingkan dengan Model From-Scratch

- Model from-scratch menghasilkan $F1 \approx 0.6884$, jauh lebih tinggi daripada kedua model zero-shot.
- Ini menunjukkan bahwa dataset Garuda Indonesia memiliki karakteristik yang cukup khusus, sehingga training spesifik pada dataset jauh lebih efektif daripada mengandalkan model multilingual umum.

4. Faktor-faktor penyebab zero-shot rendah

- Ketidaksesuaian domain pretraining dan domain dataset
- Label zero-shot berbasis entailment berbeda dari formulasi sentiment classification lokal
- Bahasa Indonesia memiliki variasi frasa dan struktur sintaks yang tidak sepenuhnya familiar bagi BART
- Model tidak di-finetune sama sekali sehingga tidak mengadaptasi pola dari dataset

e) Kesimpulan Bagian E

Evaluasi zero-shot menunjukkan bahwa model pretrained besar seperti XLM-RoBERTa Large dan BART-Large MNLI belum mampu mengungguli model Transformer yang dilatih secara khusus pada dataset Garuda Indonesia. Meskipun model pretrained memiliki cakupan bahasa yang luas dan kapasitas besar, kesenjangan domain dan ketidakhadiran fine-tuning menyebabkan performanya jauh lebih rendah ($F1$ sekitar 0.40–0.42).

Sebaliknya, model from-scratch menghasilkan performa jauh lebih baik ($F1 \approx 0.6884$) karena belajar langsung dari pola bahasa dan konteks yang spesifik dengan dataset. Hal ini mengindikasikan bahwa untuk tugas sentiment analysis pada domain yang sempit atau spesifik, fine-tuning atau training khusus pada dataset lokal memberikan hasil yang jauh lebih efektif dibandingkan zero-shot classification.

Tabel Perbandingan Akhir

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline 10 Epoch (best epoch)	0.6143	0.4009	0.4901	0.4517
Baseline 50 Epoch (best epoch)	0.7857	0.7468	0.7122	0.7188
Clean Text 50 Epoch (best epoch)	0.7000	0.6578	0.6588	0.6547
Best Hyperparameter Tuning (embed256_h4_l2_do0.1)	0.7857	0.7648	0.7715	0.7648
Zero-Shot XLM-R	0.4648	0.4101	0.4084	0.4089

Zero-Shot BART-Large MNLI	0.4930	0.4524	0.4314	0.4280
---------------------------	--------	--------	--------	--------

F. Script atau kode yang digunakan dapat di akses melalui link sebagai berikut

Link Notebook:

🔗 Tugas 2_Performance Analisis Sentimen with Transformer_PBA.ipynb

Link Notebook Cleaning: 🔗 garuda_indonesia_article_preprocess.ipynb

Link Penjelasan Cleaning dan Dataset: 📄 2025-1_Klp-11_Paper

Link Github (Laporan, Dataset, dan Notebook):

<https://github.com/jasonnho/flygaruda-sentiment-analysis/tree/main/Tugas%202>